



AIX-MARSEILLE UNIVERSITÉ
MASTER 2 INFORMATIQUE PARCOURS INGÉNIERIE DU
DÉVELOPPEMENT LOGICIEL

RAPPORT DE STAGE

**Véhicule Autonome et Connecté :
Développement des contrôleurs avec
interface graphique de gestion
d'algorithmes d'estimation**

Etudiant : Xuan Viet CONG

Tuteur : Ali ZEMOUCHE

Enseignant : Line JAKUBIEC

Année 2024/2025

Table des matières

Remerciements

Ce travail a été réalisé au Centre de Recherche en Automatique de Nancy (CRAN), au sein de l'équipe CID (Contrôle, Identification et Diagnostic), et sous la tutelle de Monsieur Ali Zemouche, que je remercie sincèrement pour m'avoir accueilli au sein de son groupe de recherche, et m'avoir permis de découvrir le monde scientifique dans un cadre stimulant.

Je remercie sincèrement mon encadrant principal, Huy Quang Nguyen, doctorant à l'IUT Henri Poincaré de Longwy, pour son accompagnement tout au long de ce stage. Ses conseils, sa disponibilité et sa patience m'ont été précieux. Grâce à son soutien, j'ai pu progresser rapidement dans un domaine qui m'était nouveau et prendre confiance dans mes capacités.

Je remercie chaleureusement l'ensemble du personnel de l'IUT de Longwy, qui a contribué à rendre ce stage agréable et productif, ainsi que mes collègues doctorants et stagiaires, avec qui j'ai partagé des échanges riches et une ambiance conviviale.

Enfin, j'adresse mes remerciements les plus sincères à ma famille et à mes proches pour leur soutien constant, leur patience et leurs encouragements, qui ont été essentiels à la réussite de ce stage et de mes études.

1 Rapport de synthèse

1.1 Introduction

Ce rapport de synthèse présente une vue d'ensemble de mon stage effectué au sein du CRAN (Centre de Recherche en Automatique de Nancy), à l'IUT de Longwy, du 1er avril au 30 septembre. L'objectif de ce rapport est de mettre en évidence ce que j'ai appris au cours de cette expérience, ainsi que le recul pris vis-à-vis des missions réalisées et des connaissances développées.

Afin de donner une vision claire et structurée de mon stage, ce rapport est organisé en plusieurs parties :

- **Présentation de l'Institution et du Service d'Accueil** : une description du CRAN et de l'IUT de Longwy, mettant en avant leur rôle dans l'enseignement, la recherche et l'innovation scientifique, ainsi que le service ou laboratoire où s'est déroulé mon stage.
- **Sujet du Stage** : une définition précise de la thématique de stage, de ses objectifs initiaux et du contexte scientifique ou technique.
- **Description de l'Existant** : l'état initial avant mon intervention, mettant en lumière les problématiques rencontrées ou les besoins identifiés.
- **Travail Réalisé et Solutions Apportées** : une présentation des différentes étapes de mon travail, des outils employés, et des résultats obtenus.
- **Méthodologie Employée** : une explication des méthodes de recherche, d'analyse ou de développement utilisées au cours du stage.

1.2 Présentation de l'Entreprise

1.2.1 Le CRAN

Le stage se déroule au sein du Centre de recherche en automatique de Nancy (CRAN) qui a été créé en 1980. Le CRAN est une unité mixte de recherche (UMR 7039) commune à l'Université de Lorraine et au Centre national de recherche scientifique (CNRS).

Le laboratoire est implanté sur trois sites principaux : Nancy, Épinal et Longwy.



FIGURE 1 – Plan d'accès et implantation du CRAN en France.

Au 1er janvier 2024, le CRAN comptait environ 250 membres :

- 120 chercheurs ou enseignants-chercheurs (dont 8 chercheurs CNRS),
- des chercheurs de l'Institut de Cancérologie de Lorraine (ICL) du Centre Hospitalier Régional Universitaire (CHRU) ou d'organismes externes,
- 5 chercheurs émérites,
- 27 personnels administratifs et techniques,
- Environ 100 doctorants, ATER, post-doctorants et chercheurs invités.

Le laboratoire est organisé en trois départements scientifiques :

Département	Domaines principaux
Contrôle, Identification et Diagnostic (CID)	Automatique, systèmes dynamiques
Modélisation, Pilotage et Sûreté des systèmes Industriels (MPSI)	Automatique, productique, génie informatique
Biologie, Signaux et Systèmes en Cancérologie et Neurosciences (BioSiS)	Biologie, neurologie, traitement du signal et de l'image

TABLE 1 – Les différents départements du CRAN.

1.2.2 Le site de Longwy

Mon stage s'est déroulé sur le site de l'IUT de Longwy, au sein du département Contrôle, Identification et Diagnostic (CID). Ce département compte actuellement 24 membres, dont :

- 4 professeurs des universités dont 1 émérite,
- 5 maîtres de conférences,
- 1 ingénieur de recherche CNRS,
- 2 post-doctorants,
- 1 ATER,
- 4 doctorants,
- 8 stagiaires en master.

Les thématiques de recherche du département portent sur l'estimation, la commande et le diagnostic pour les systèmes avec des complexités spécifiques :

- systèmes interconnectés en réseaux,
- systèmes de grande dimension,
- systèmes avec des paramètres incertains,
- systèmes stochastiques.

Ces recherches trouvent des applications dans des domaines variés :

- véhicules autonomes et transports,
- environnement et bio-procédés,
- énergie, l'hydrogène (H_2) en particulière,
- communications.

1.3 Définition du Sujet du Stage

1.3.1 Cahier des Charges Initial

Le stage avait pour objectif de développer et valider des méthodes avancées d'observation d'état appliquées aux véhicules autonomes. Le cahier des charges initial incluait l'étude et la mise en œuvre de deux approches d'observations pour les systèmes non linéaires : les observateurs à grand gain (High-Gain Observers, HGO) et les observateurs à intervalle de temps fini (Fixed-Time LPV Observers, FLPV).

L'environnement de travail choisi était le simulateur CARLA, un environnement de simulation 3D *open source* très utilisé par la communauté académique et scientifique. CARLA permet de reproduire des situations de circulation réelles (autoroutes, intersections, ...) et constitue ainsi une plateforme idéale pour exposer l'observateur à des situations proches de la réalité.



FIGURE 2 – Simulateur 3D CARLA.

Le stage devait se dérouler en plusieurs étapes :

- Etude bibliographique et théorique : analyse des principes des observateurs HGO et FLPV et revue des défis en conduite autonome.
- Modélisation et implémentation : modélisation des véhicules dans CARLA, puis implémentation des algorithmes d'observation HGO/FLPV en Python/C++.
- Interface utilisateur : développement d'un outil de visualisation permettant de configurer les observateurs, de suivre leur comportement en temps réel et d'analyser les résultats. Le cahier des charges initial envisageait une application de type Qt.
- Simulation complexe : La mise en place de scénarios complexes de simulation (suivi sur autoroute, navigation aux intersections, ...) afin d'évaluer la précision et la robustesse des méthodes étudiées.
- Documentation et recommandations : rédaction d'un rapport scientifique comprenant un guide méthodologique et les recommandations pour des applications réelles futures.

1.3.2 Évolution du Sujet

Au cours du stage, et après consultation des responsables du CRAN, le projet a été évolué afin de mieux s'adapter aux ressources et moyens techniques disponibles sur le site

de Longwy et d'élargir la portée expérimentale du projet. Deux changements majeurs ont été apportés : le choix de l'environnement de test et la nature de l'interface utilisateur.

Environnement expérimental :

Premièrement, au lieu de s'appuyer uniquement sur le simulateur CARLA, le stage a exploité les solutions de la société Quanser, à savoir le simulateur QLab et le véhicule physique QCar2. Le simulateur QLab offre une plateforme plus directement orientée vers l'enseignement et la recherche appliquée en automatique et en robotique. Il permet de tester rapidement des algorithmes dans un environnement virtuel simplifié mais réaliste, facilitant une itération rapide entre la théorie et pratique. Quant à QCar2, il s'agit d'un modèle de véhicule physique équipé de multiples capteurs (caméras, LIDAR, IMU, ...) et d'un système embarqué de calcul fonctionnant sous Ubuntu, ce qui facilite l'intégration avec des environnements logiciels tels que les bibliothèques de contrôle et de vision disponibles. Il est donc un support idéal pour passer de la simulation aux tests réels, tout en conservant un environnement contrôlé et sécurisé.



FIGURE 3 – Logo Quanser



FIGURE 4 – Simulateur QLab



FIGURE 5 – QCar2

Ce passage de CARLA à QLabS et QCar2 a élargi la dimension expérimentale du projet, en validant les observateurs non seulement dans des scénarios virtuels mais aussi sur un véhicule physique.

Interface utilisateur :

Deuxièmement, le développement de l'interface utilisateur a évolué vers une application web. Plutôt qu'une application de bureau locale en Qt, l'interface est conçue comme une application web interactive, accessible depuis un navigateur.

Cette solution présente plusieurs avantages :

- Portabilité : Une fois qu'un navigateur moderne est installé, l'application est utilisable sur n'importe quel poste de travail sans installation complexe.
- Déploiement et partage : elle peut être hébergée sur un serveur dans un réseau local, permettant l'accès simultané par plusieurs utilisateurs.

L'application web permet de configurer les paramètres des observateurs, de suivre en temps réel les estimations produites et d'analyser visuellement leurs performances sur des scénarios simulés ou réels.

Aspect	Cahier des charges initial	Évolution du sujet
Environnement de test	Simulateur CARLA uniquement	QLabs (simulation) + QCar2 (véhicule physique)
Méthodes étudiées	Observateurs HGO et FLPV	Observateurs HGO et FLPV (inchangé)
Interface utilisateur	Application de bureau locale (Qt)	Application web interactive accessible via un navigateur
Validation	Scénarios simulés (autoroute, intersections, etc.)	Validation hybride : simulation + expérimentation réelle sur véhicule

TABLE 2 – Tableau comparatif : Cahier des charges initial vs Évolution du sujet

1.4 Description de l'Existant

Avant le début de mon stage, les travaux menés portaient principalement sur l'étude théorique des méthodes d'observation d'état. Les principes des observateurs HGO et des observateurs FLPV avaient été largement étudiés sur le plan mathématique, et des résultats intéressants avaient déjà été obtenus au niveau de la modélisation et de l'analyse de leurs propriétés.

Cependant, ces travaux restaient majoritairement confinés à des analyses théoriques et à des simulations numériques limitées, souvent réalisées dans des environnements de calcul simplifiés (scripts MATLAB...). Il manquait une étape essentielle : la validation expérimentale dans des environnements plus proches des conditions réelles.

Pour cela, deux plateformes étaient disponibles au laboratoire :

- Le simulateur QLab de Quanser, permettant de recréer un environnement virtuel réaliste où un véhicule autonome peut interagir avec son environnement. Cet outil offre la possibilité de tester des algorithmes dans des conditions variées comme le trafic, les obstacles, ...
- Le véhicule physique QCar2 de Quanser, un modèle de voiture connectée équipé de multiples capteurs (caméra, LIDAR, IMU, ...) et d'un système informatique intégré, est une plateforme pour passer de la simulation aux tests en conditions réelles et évaluer des observateurs dans des conditions physiques contrôlées.

Par conséquent, le cadre actuel présente un écart entre des résultats théoriques prometteurs et un manque de validation dans des environnements plus réalistes. L'intégration des observateurs HGO et FLPV n'a pas encore été implémentée dans QLab ni dans QCar2. De plus, il n'existe aucune interface utilisateur permettant d'avoir un contrôle intuitif des observateurs et l'analyse de leurs performances.

Ce stage vise donc à combler le fossé entre théorique et pratique en assurant la mise en œuvre, l'intégration et la validation des observateurs sur les plateformes existantes.

1.5 Travail Effectué

Cette section présente et met en valeur le travail effectué. La description technique détaillée du travail se trouve dans le rapport technique.

1.5.1 Planification (Plan directeur)

Plan initial

Au début du stage, un plan directeur a été élaboré pour organiser le travail et prévoir les livrables associés. Ce document explique brièvement le contexte du stage et propose un plan couvrant la période d’avril à septembre 2025.

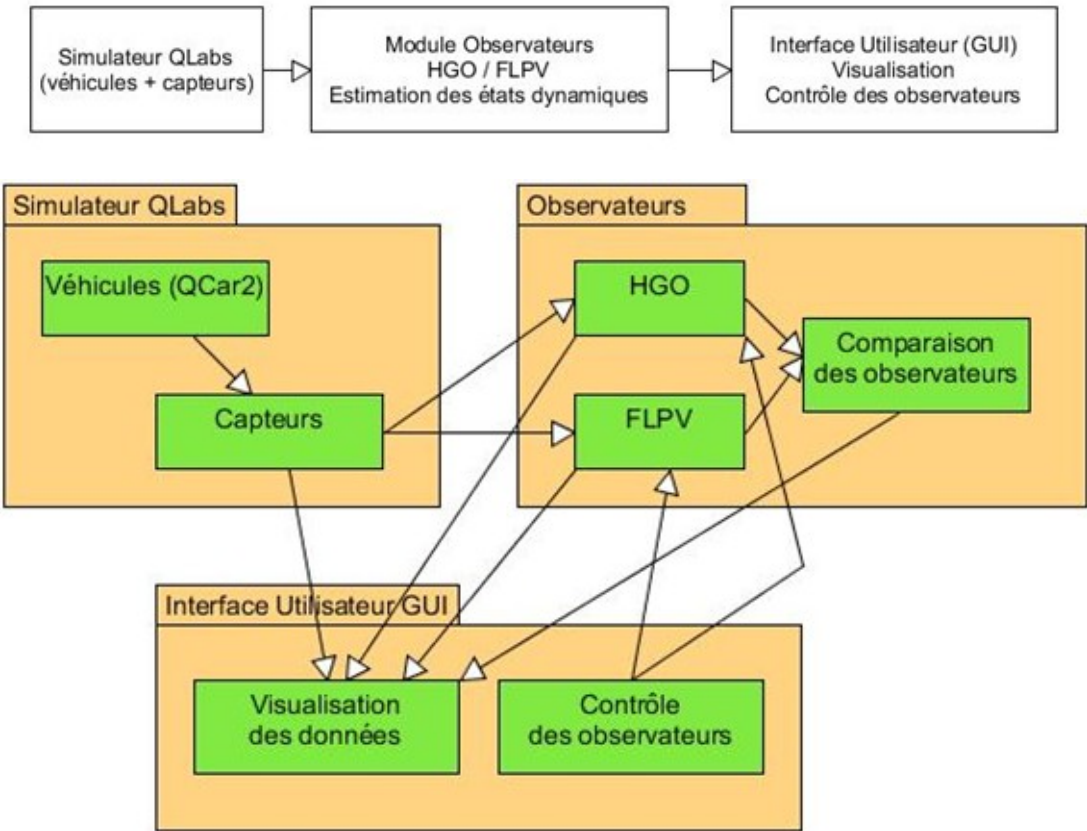


FIGURE 6 – Diagrammes de contexte.

Les étapes principales prévues sont présentées dans le tableau ci-dessous :

Période	Tâche principale	Livrables
Avril 2025	Prise en main de QLABS et ROS2	Rapport de découverte et configuration
Mai 2025	Prise en main du véhicule QCar2 physique et tests de la connectivité	Tests et configuration réseau
Juin 2025	Intégration des observateurs avec QLABS	Module de test fonctionnel
Juillet 2025	Développement de l'interface utilisateur GUI	Prototype GUI
Août 2025	Tests dans scénarios simulés et physiques	Résultats comparatifs, logs
Septembre 2025	Analyse des performances, documentation complète	Rapport final, guide méthodologique

TABLE 3 – Phasage du stage.

Evolution

Au fur à mesure de l'avancement, mes responsabilités ont considérablement évoluées. Initialement centré sur l'intégration des observateurs, mon rôle a évolué vers des missions plus transversales liées à l'ingénierie logicielle. En effet, l'équipe dans laquelle j'ai travaillé disposait de compétences en automatique et en théorie des systèmes, mais avait peu d'expérience en informatique et en développement collaboratif.

Grâce à mon profil en ingénierie du développement logiciel, j'ai pris en charge :

- Le développement du véhicule autonome (simulation dans QLABS et l'implémentation sur le QCar2 physique), afin de fournir une base prête à l'emploi sur laquelle les chercheurs pouvaient appliquer leurs algorithmes et observateurs.
- Un rôle de consultant informatique, fournissant un support technique quotidien à l'équipe pour résoudre des problèmes liés aux environnements, aux configurations réseau ou à la programmation.
- De mise en place d'une méthodologie de développement structurée, adaptée à une équipe peu familière avec les pratiques logicielles modernes

1.5.2 Prise en main des environnements techniques

Avant de commencer le développement, une étape importante est de comprendre et de se familiariser avec l'environnement logiciel et matériel utilisé pour le projet, notamment QLABS et ROS2, qui sont les plateformes de simulation et de communication des véhicules autonomes.

Présentation de ROS2

ROS2 (Robot Operating System 2) est un *middleware* largement utilisé dans la recherche et l'industrie robotiques. Il ne s'agit pas d'un système d'exploitation au sens classique, mais d'un ensemble de bibliothèques et d'outils facilitant le développement d'applications robotiques.



FIGURE 7 – Logo ROS

Son rôle principal est d'assurer la communication entre les différents modules logiciels grâce à un mécanisme basé sur :

- des nœuds : des processus indépendants représentant un capteur, un contrôleur ou un algorithme,
- des topics : les canaux de communication qui permettent l'échange de messages et des données en continu entre les nœuds,
- des services : les appels synchrones entre les nœuds.
- ...

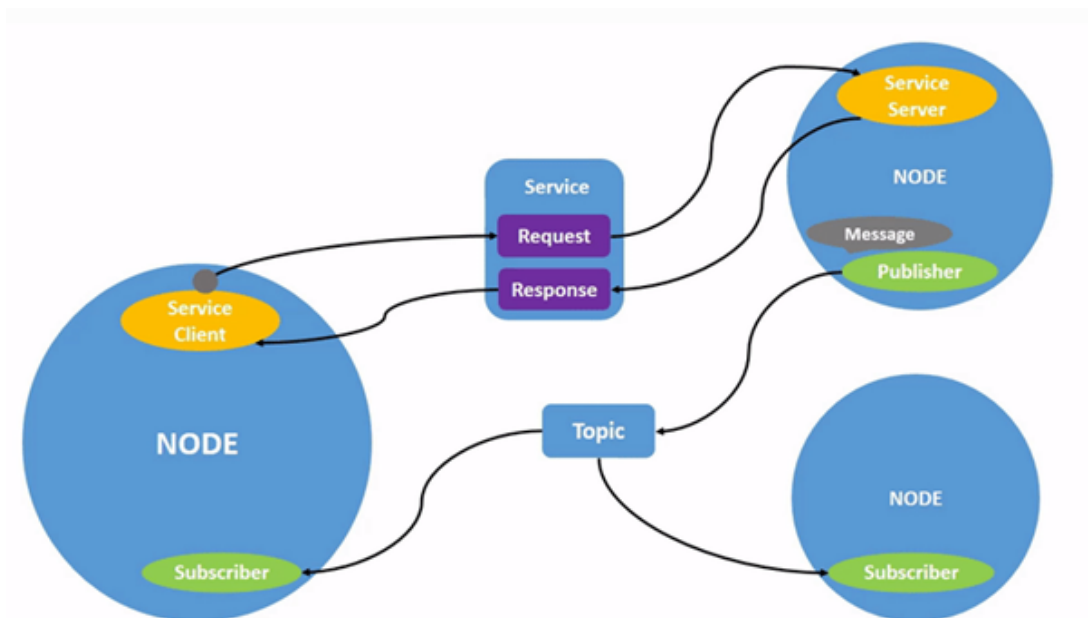


FIGURE 8 – Architecture de communication de base de ROS

Grâce à sa modularité, sa portabilité et sa capacité à s'intégrer à la fois aux robots physiques et aux environnements de simulation, ROS2 devient un outil essentiel pour le développement de véhicules autonomes.

Présentation de QLab

Interface utilisateur et workspaces

L'interface graphique de QLab propose une navigation dans différents produits (robots développés par Quanser, ici on choisit QCar2) et différents *workspaces* (espaces de simulation prédéfinis). Les différents workspaces correspondent à des environnements réalistes (zones urbaines, intersections, autoroutes, ...) dans lesquels il est possible d'instancier des objets virtuels.

Les icônes de l'interface utilisateur en haut de l'écran fournissent des commandes et des informations sur la caméra à l'utilisateur.

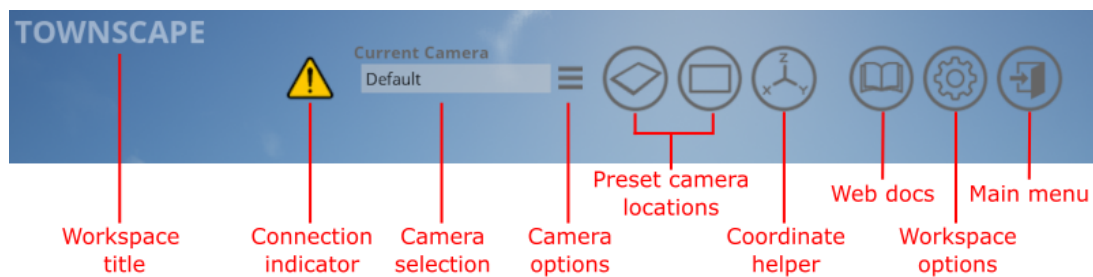


FIGURE 9 – Interface utilisateur QLab



FIGURE 10 – Exemple : Le workspace Cityscape

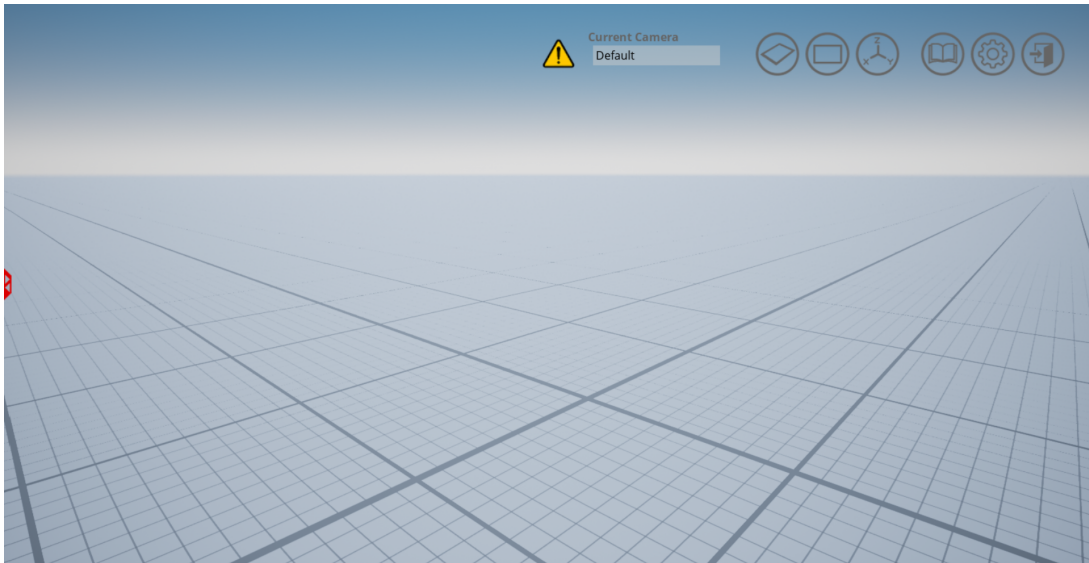


FIGURE 11 – Exemple : Le workspace Plane

L'utilisateur peut :

- sélectionner un workspace et visualiser la scène 3D
- instancier des acteurs (véhicules, obstacles, piétons, ...)
- interagir avec la simulation (navigation libre dans l'environnement, caméra paramétrée, configuration de la scène, ...)

Bibliothèques système

D'après leur *Design Philosophy* (philosophie de conception), QLab n'est pas conçu pour être utilisé uniquement via l'interface utilisateur. Plutôt que d'interagir directement avec QLab, Quanser fournit une bibliothèque dédiée en Python ainsi qu'un SDK (*Software Development Kit*) permettant aux développeurs d'instancier et de contrôler des éléments de simulation.

La structure de ces bibliothèques est :

- **QLabs Core** : fournir les fonctions de base pour initialiser la communication avec le simulateur et gérer les connexions.
- **System** : gérer l'interface utilisateur et le fenêtre de QLab.
- **Real-Time** : lancer ou arrêter les modèles en temps réel conçu pour s'interfacer avec QLab
- **Collection d'acteurs** : représenter des objets statiques ou dynamiques comme les véhicules (QCar2), les obstacles, les panneaux de signalisation, les piétons, ..., et les instancier dans l'environnement.
- **Quanser SDK** : propose un ensemble d'outils et d'API facilitant le développement utilisant QLab ou véhicule physique. Il fournit des pilotes de périphériques, des API en C et Python et des bibliothèques de communication et d'interfaces matérielles.
- etc.

Présentation de QCar2

Le QCar2 est une plateforme mobile développée par Quanser pour l'enseignement et la recherche sur les véhicules autonomes. Il s'agit d'un véhicule connecté, à l'échelle 1/10, équipé d'un ensemble de multiples capteurs et d'actionneurs, et d'un puissant système embarqué, permettant la mise en œuvre de scénarios réalistes de conduite autonome et de développer, tester et valider des algorithmes d'observation.

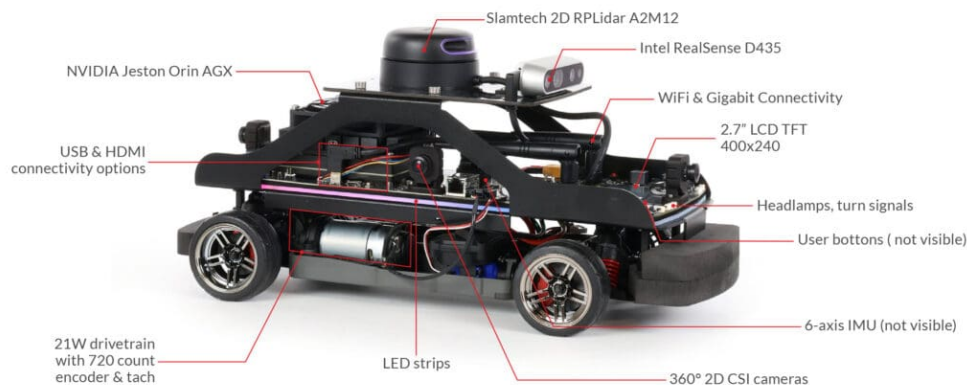


FIGURE 12 – Le QCar2 avec ses différents éléments.

- **Unité de calcul** : NVIDIA Jetson Orin AGX, avec un ordinateur embarqué fonctionnant sous Ubuntu.
- **Capteurs de perception** : un LIDAR qui scanne l'environnement en 2D, une caméra 3D (Intel RealSense) pour la vision de profondeur, ainsi que quatre caméras panoramiques pour une vue à 360°.
- **Capteurs de navigation** : un capteur IMU 6 axes qui mesure les mouvements (accélérations et rotations) et des encodeurs de roue pour suivre précisément les déplacements.
- **Connectivité** : le véhicule peut communiquer via Wi-Fi ou câble Ethernet, et dispose de ports USB/HDMI pour le développement.
- **Interface** : un petit écran LCD, des boutons utilisateurs, ainsi que des phares et clignotants pour simuler un vrai véhicule.

Cette combinaison de capteurs et de puissance de calcul rend le QCar2 particulièrement adapté pour expérimenter des scénarios réalistes

Simulation du QCar2 dans QLabS

Une partie importante a consisté à comprendre et mettre en place la simulation du véhicule QCar2 dans QLabS. J'ai consacré du temps à étudier le code source des bibliothèques QLabS et ROS2 fournies par Quanser. Cela m'a permis de comprendre en profondeur le fonctionnement interne du simulateur et de son intégration avec ROS2.

Le processus se déroule en plusieurs étapes :

- **Instancier l'acteur QCar2** : dans QLabS, on instancie un acteur représentant le véhicule QCar2 dans l'environnement virtuel.
- **Démarrage du modèle temps réel (rt model)** : une fois l'acteur instancié, un modèle temps réel est lancé. Ce modèle simule le comportement du véhicule et génère en continu les flux de données issus des capteurs (caméras , LIDAR, ...)
- **Emission locale des données** : ces flux de données sont simulés sur les ports locaux de la machine exécutant l'application QLabS. Chaque capteur virtuel produit un flux comparable à celui du QCar2 réel.
- **Bibliothèque ROS2** : une bibliothèque ROS2 développée et fournie par Quanser, permet de récupérer ces données simulées et de les publier sur des topics ROS2 correspondants. Pour l'utilisateur, le QCar2 virtuel apparaît comme un vrai robot ROS2 avec ces capteurs accessibles en temps réel.

Grâce à cette architecture, QLabS offre une simulation fidèle et intégrée du QCar2, permettant de développer et tester des algorithmes avant de les déployer sur le véhicule physique.

Utilisation de Docker pour ROS2 et QLabS

Docker est utilisé comme environnement principal de développement et d'exécution.

Concrètement :

- Les *builds* (compilation du code, génération des packages ROS2),
- Le lancement des nœuds ROS2 (localisation, perception, contrôle),
- L'exécution des scénarios de simulation et de communication avec QLabS/QCar2

ont été réalisés à l'intérieur d'un conteneur Docker préconfiguré.

Les avantages sont multiples :

- **Reproductibilité, et isolation** : l'environnement est identique sur toutes les machines, ce qui évite les problèmes liés aux versions de bibliothèques ou aux différences de configuration système. Les dépendances logicielles (ROS2, Python, ...) restent isolées du système hôte, évitant les conflits.
- **Portabilité** : le conteneur peut être déployé facilement sur n'importe quel poste.

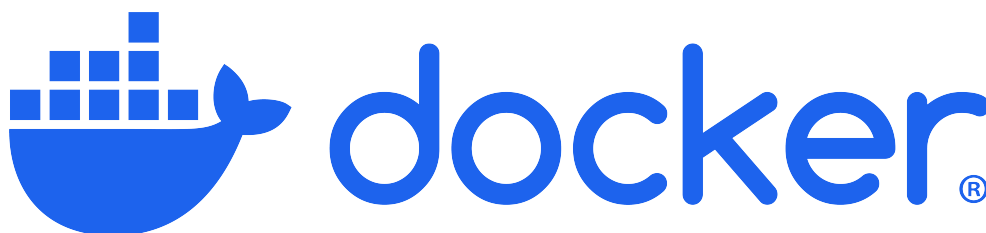


FIGURE 13 – Docker

1.5.3 Développement du véhicule autonome

Une grande partie de mon stage consistait à développer un véhicule autonome fonctionnel, utilisable à la fois dans l'environnement de simulation QLab et sur le véhicule physique QCar2. L'objectif était de créer une plateforme capable de se déplacer de manière autonome, afin que les chercheurs puissent appliquer et tester leurs algorithmes d'observation.

Environnement simulé

Dans QLab, le véhicule virtuel QCar2 est instancié dans workspace « Plan », avec l'environnement généré par le script fourni par Quanser.. Le modèle temps réel est également lancé, simulant la dynamique du véhicule et générant en continu des données issues de ses capteurs (lidar, caméras, GPS, IMU). Ces flux sont ensuite publiés dans ROS2 sous forme de topics.

Suivi de trajectoire avec Stanley Controller

Le suivi de trajectoire est un élément central de la conduite autonome. Pour cette tâche, j'ai utilisé le Stanley Controller, dont l'implémentation est fournie par Quanser pour le QCar2.

Le Stanley Controller est un algorithme classique de robotique mobile, initialement développé à Stanford pour la voiture autonome "Stanley", vainqueur du DARPA Grand Challenge (2005).



FIGURE 14 – La voiture autonome Stanley

Il repose sur le principe suivant :

- Le contrôleur calcule en continu l’erreur latérale (distance entre la position actuelle du véhicule et la trajectoire de référence).
- Il prend également en compte l’angle d’orientation du véhicule par rapport à la trajectoire.
- En combinant ces deux informations, il génère une commande de direction qui corrige la trajectoire du véhicule tout en assurant sa stabilité.

Dans le QCar2, l’implémentation fournie par Quanser permet un suivi robuste et précis des trajectoires planifiées dans QLab. Mon rôle a consisté à intégrer ce contrôleur dans l’architecture ROS2 et à configurer les paramètres nécessaires.

Localisation

Pour estimer la position du véhicule, j’ai utilisé l’approche combinant le GPS avec un filtre de fusion de données fourni par Quanser.

GPS de Quanser basé sur le LIDAR

En explorant le code source de la bibliothèque fournie par Quanser, j’ai constaté que la classe GPS n’exploite pas directement un capteur GPS physique classique. En réalité, elle lance un modèle temps réel spécifique (fichier `.rt-linux_qcar2`) qui génère une estimation de position à partir des données du LIDAR.

L’implémentation interne de ce modèle n’est pas documentée, mais en analysant les appels de fonctions et les flux de données, j’ai identifié que la position est calculée à partir d’un traitement du LIDAR qui est une approche de *Scan Matching* : les nuages de points générés par le LIDAR sont comparés pour estimer les déplacements du véhicule.

Ce “GPS” proposé par Quanser correspond en réalité à une estimation de la position, construite à partir des mesures du LIDAR, et imite le comportement d’un GPS standard.

EKF

Cet algorithme de filtrage EKF (*Extended Kalman Filter*, ou filtre de Kalman étendu) est particulièrement adapté aux systèmes non linéaires comme le QCar2. Il repose sur deux étapes principales :

1. Prédiction : l’état du véhicule (position, vitesse, orientation) est estimé à partir du modèle dynamique et des commandes appliquées (accélération, vitesse).
2. Mise à jour : cette estimation est corrigée par les mesures issues du “GPS”.

L’EKF combine les avantages de plusieurs sources :

- le “GPS” Quanser,
- le modèle dynamique, qui fournit une estimation prédictive basée sur les commandes mais qui peut dériver avec le temps.

Évitement d'obstacles simplifié

En plus du suivi de trajectoire et de la localisation, j'ai mis en place un système simplifié d'évitement d'obstacles basé sur les données LIDAR.

Le lidar du QCar2 produit un nuage de points représentant l'environnement proche. J'ai utilisé ces données pour construire une *Occupancy Grid* (carte d'occupation), c'est-à-dire une grille qui indique quelles zones sont libres et quelles zones sont occupées par des obstacles.

- Les zones grises représentent l'espace libre.
- Les pixels blancs correspondent aux cellules identifiées comme occupées (obstacles).
- Les segments bleus représentent les mesures du LIDAR, projetées dans l'espace pour détecter les obstacles autour du véhicule.

Cette carte constitue la base pour la perception de l'environnement : les données LIDAR fournissent la grille d'occupation, qui est ensuite utilisée par l'algorithme d'évitement pour adapter la trajectoire du véhicule.

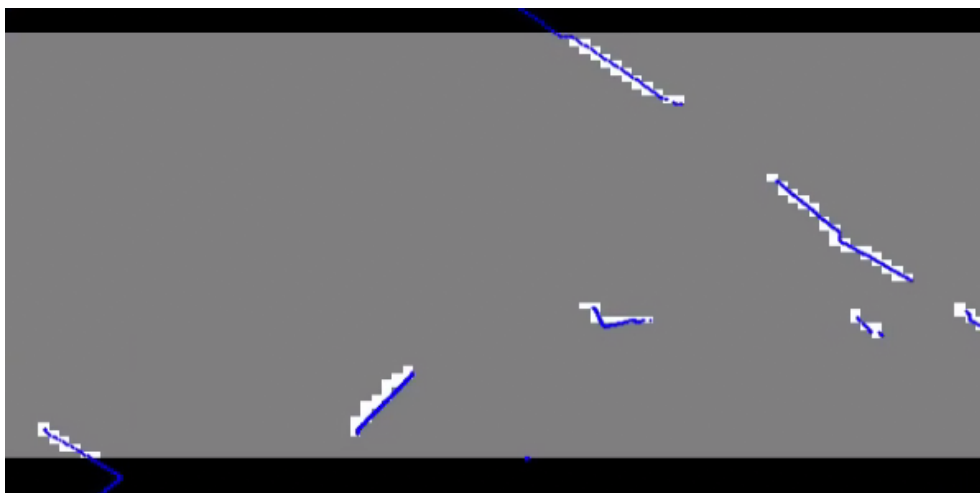


FIGURE 15 – Occupancy grid générée à partir des données LIDAR

L'algorithme, simplifié pour rester compatible avec les ressources disponibles, fonctionne ainsi :

- génération de la grille à partir des données lidar,
- détection des cellules occupées devant le véhicule,
- modification locale de la trajectoire de référence afin d'éviter la zone détectée.

Architecture logicielle

Le développement du véhicule autonome a reposé sur une architecture modulaire ROS2, dans laquelle chaque composant est implémenté sous forme de nœud indépendant. Cette

approche garantit la flexibilité et facilite l'intégration d'algorithmes supplémentaires. L'architecture inclut :

- des nœuds de localisation (GPS + EKF),
- des nœuds de perception (Occupancy Grid à partir du lidar),
- des nœuds de contrôle (suivi de trajectoire avec Stanley Controller).

1.5.4 Support et rôle de consultant informatique

En plus du développement technique, une part importante de mon travail est consacrée à un rôle de support informatique au sein de l'équipe. Le département CID possède des compétences en automatique et en théorie des systèmes, mais l'expérience des membres en développement informatique, de gestion de code et de configuration d'environnement complexes est limitée.

Dans ce contexte, j'ai joué le rôle de consultant informatique. Mes contributions ont été :

- l'installation et la configuration de QLab et ROS2 sur les différentes machines de l'équipe, incluant la gestion des dépendances, des environnements virtuels...,
- l'assistance dans le débogage des problèmes rencontrés lors de l'utilisation des bibliothèques Quanser ou des modules ROS2,
- l'accompagnement des membres de l'équipe dans la prise en main de Git, de QLab et de ROS2.

1.5.5 Gestion du dépôt Git et stratégie de développement

Dès le début, il est apparu que la gestion du code source constituait un point faible de l'organisation. Chaque membre de l'équipe travaillait de manière indépendante, avec peu de coordination. Cette approche rendait difficile l'obtention de résultats et l'intégration de contributions multiples.



FIGURE 16 – Git et GitHub.

J'ai donc proposé et mis en œuvre une méthodologie structurée de gestion du code source, basée sur Git et GitHub. Le choix s'est porté sur une stratégie de *Trunk-Based Development* (TBD), adaptée à une équipe pluridisciplinaire où la priorité est la simplicité et la rapidité d'intégration.

Les principes clés mis en place sont :

- chaque membre de l'équipe développe sur une branch temporaire, dédiée à une tâche spécifique,
- une fois le développement terminé et validé, la branche est fusionnée dans la branche principale `main` via une *Pull Request*,
- l'utilisation de GitHub Project pour organiser les tâches, assigner des responsabilités et suivre l'avancement.

Afin de rendre cette méthodologie accessible à tous, j'ai rédigé un guide détaillé dans le fichier `README` du dépôt, incluant des instructions pas à pas sur :

- la création de branches,
- la bonne pratique des commits,
- la gestion des *Pull Requests*,
- l'utilisation de GitHub Project pour la planification des tâches.

L'adoption de cette stratégie a permis d'instaurer une méthodologie collaborative claire et partagée. L'équipe a gagné en organisation et en traçabilité, et les nouveaux arrivants peuvent rejoindre le projet sans difficulté grâce à la documentation centralisée.



FIGURE 17 – Le fichier README.

1.5.6 Développement de l'interface utilisateur

Afin de faciliter l'utilisation du QCar2 et l'évaluation des observateurs, j'ai développé une interface utilisateur moderne sous forme d'une application Web. L'objectif était de rendre les expérimentations accessibles et intuitives, tout en garantissant une intégration fluide avec l'infrastructure logicielle existante.

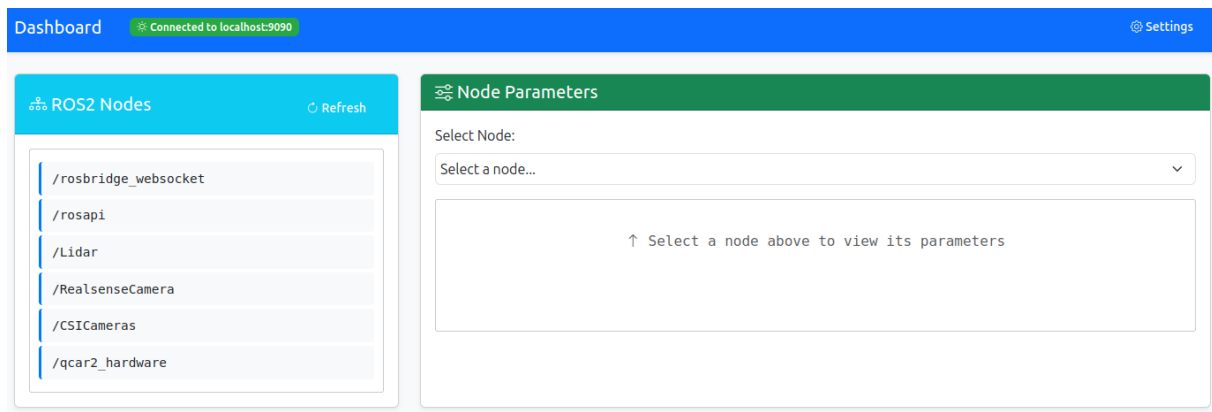


FIGURE 18 – Prototype d'interface utilisateur web.

Architecture technique

L'application web repose sur une architecture simple mais efficace :

- un serveur Express.js (Node.js) pour héberger et servir l'application web,
- la suite `rosbridge-suite` et la bibliothèque `roslibjs` de Robot Web Tools
 - `rosbridge-suite` agit comme un serveur intermédiaire exposant ROS2 via WebSocket,
 - `roslibjs`, utilisée côté navigateur, permet de communiquer avec `rosbridge` et de manipuler les éléments ROS2 (topics, paramètres, ...).

Grâce à cette architecture, l'application web peut :

- s'abonner à des topics ROS2 pour interagir avec le QCar2,
- modifier les paramètres des nœuds en cours d'exécution,
- recevoir et afficher en temps réel les flux de données du robot (capteurs, observateurs, ...).

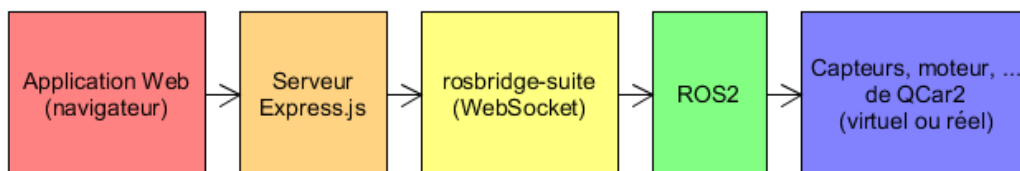


FIGURE 19 – Architecture de communication Application Web - ROS2 - QCar2.

1.6 Méthodologie

1.6.1 Organisation du travail

Le programme de stage s'appuie sur un plan directeur prédéfini, qui définit les principales étapes : familiarisation avec l'environnement (ROS2, QLabS, QCar2), développement du véhicule autonome, intégration des observateurs et mise en œuvre des outils d'aide à l'utilisation de la plateforme. Ce plan sert de référence, mais est régulièrement ajusté en fonction

- des priorités de l'équipe,
- des difficultés rencontrées.

L'approche utilisée est itérative : chaque nouvelle fonctionnalité (véhicule autonome, interface web, ...) est développée et validée progressivement, par simulation dans QLabS et QCar2 physique.

1.6.2 Méthodologie de développement logiciel

Mon expérience en ingénierie logicielle m'a permis d'appliquer des méthodes structurées au sein de l'équipe :

- utilisation de Git et GitHub pour le contrôle de version, avec une stratégie de *Trunk-Based Development* facilitant l'intégration rapide des contributions,
- utilisation de branches temporaires et de Pull Requests pour garantir la qualité du code,
- documentation systématique des procédures dans un fichier README et mise en place d'un GitHub Project pour le suivi des tâches.

1.6.3 Collaboration et support

Un aspect important de la méthodologie a été la collaboration avec les membres de l'équipe. J'ai joué le rôle de référent informatique, en aidant à la configuration des environnements et en résolvant les problèmes techniques. Cette approche a amélioré la performance de l'équipe et contribué à renforcer la collaboration au sein du département.

1.6.4 Outils utilisés

Le projet a reposé sur un ensemble d'outils logiciels et matériels :

- **Outils de développement logiciel :**
 - Git et GitHub (gestion de versions et suivi de projet),
 - Visual Studio Code,
 - Python (langage principal pour ROS2),
 - Docker,

- Node.js et Express.js (serveur de l'application web),
- roslibjs et rosbridge-suite (Robot Web Tools) pour la communication entre application web et ROS2.
- **Outils de simulation et robotique :**
 - QLab (environnement de simulation réaliste développé par Quanser),
 - QCar2 (plateforme robotique physique équipée de capteurs),
 - ROS2 (*middleware* de robotique pour la gestion des nœuds, topics, services).

1.7 Difficultés rencontrées

1.7.1 Complexité des environnements logiciels

L'une des premières difficultés a été la complexité de l'environnement ROS2 et des bibliothèques fournies par Quanser. L'installation et la configuration de ROS2 (dépendances, compatibilité avec le système d'exploitation) ont demandé un travail important de compréhension et de documentation.

Solution retenue : j'ai mis en place des environnements reproductibles et rédigé un guide pratique pour l'équipe, afin que chacun puisse configurer son poste de travail rapidement et éviter de répéter les mêmes erreurs.

1.7.2 Documentation limitée des bibliothèques Quanser

Les bibliothèques QLabS et QCar2 fournies par Quanser sont puissantes, mais leur documentation est parfois limitée. La compréhension de leur fonctionnement interne (exemple : GPS basé sur LIDAR et *scan matching*) n'était pas toujours claire.

Solution retenue : j'ai analysé le code source fourni et expérimenté directement dans QLabS pour mieux comprendre le comportement des classes. J'ai ensuite partagé ces connaissances avec l'équipe, ce qui a permis de débloquer plusieurs travaux.

1.7.3 Différences de compétences au sein du groupe

Le projet s'est déroulé au sein d'une équipe aux compétences très variées : une forte expertise théorique en automatique, mais une expérience limitée en informatique et en gestion logicielle. De mon côté, ma formation et mon expérience étaient principalement orientées vers l'ingénierie logicielle, avec peu d'expérience en automatique et en robotique.

Cela a représenté un double défi :

- aider mes collègues dans les aspects informatiques (développement logiciel, gestion de code, ...),
- tout en m'appropriant rapidement les notions de contrôle, de capteurs et de robotique mobile nécessaires pour comprendre le projet et contribuer efficacement.

Solution retenue : j'ai joué un rôle de référent informatique en formant mes collègues à Git, ROS2 et QLabS, et en instaurant une stratégie de développement claire (*Trunk-Based Development*, GitHub Project). En parallèle, j'ai investi du temps dans l'auto-apprentissage de la robotique et de l'automatique (lecture de documentation, étude des bibliothèques Quanser, échanges avec les chercheurs), ce qui m'a permis de combler progressivement mes lacunes et de contribuer au développement du véhicule autonome.

1.8 Conclusion

1.8.1 Bilan du stage

Ce stage m’a permis de travailler sur un projet informatique, robotique et automatique, dans un environnement de recherche exigeant.

- Points positifs :
 - J’ai contribué de manière significative à la mise en place d’un développement collaboratif : gestion du dépôt Git, mise en place d’une stratégie de développement (*Trunk-Based Development*), rédaction d’un guide d’utilisation pour les membres de l’équipe.
 - J’ai acquis beaucoup d’expérience pratique avec ROS2 et l’écosystème Quanser (QLabs, QCar2)
 - J’ai développé des modules pour le véhicule autonome : construction d’un `OccupancyGrid` avec LIDAR, intégration d’un contrôleur Stanley, logique simplifiée d’évitement d’obstacles.
- Difficultés rencontrées :
 - Mon manque d’expérience initiale en automatique et en robotique m’a obligé à approfondir de nouveaux concepts (localisation, contrôleurs, observateurs d’état).
 - Certains problèmes logiciels (par exemple la compatibilité des versions, ...) nécessitent du temps pour être compris et corrigés.

1.8.2 Perspectives

Le travail effectué constitue une première étape vers une intégration complète des observateurs théoriques développés au CRAN dans un environnement expérimental. Plusieurs améliorations sont envisagées :

- Implémenter et tester les observateurs HGO et FLPV dans QLab, puis sur le QCar2 physique.
- Améliorer l’évitement d’obstacles en remplaçant la logique simplifiée actuelle par une planification plus avancée.
- Enrichir l’application web et ajouter plus de fonctionnalités, comme gérer plusieurs robots à la fois ...

1.8.3 Conclusion générale

Ce stage au CRAN a été pour moi une expérience très enrichissante, tant sur le plan professionnel que personnel. J’ai découvert un domaine nouveau - l’automatique et la robotique - dans lequel je n’avais que très peu d’expérience au départ. Cela m’a demandé un effort d’adaptation et d’apprentissage rapide, mais cette difficulté s’est transformée en une opportunité de développer de nouvelles compétences.

J'ai particulièrement apprécié de pouvoir mettre à profit mes connaissances en développement logiciel pour répondre à des besoins concrets de l'équipe : mise en place d'une méthodologie de travail collaborative, gestion d'un dépôt Git, conception d'une interface web intuitive. Jouer un rôle de référent informatique dans une équipe de chercheurs m'a permis de mesurer la valeur de mes compétences et de prendre confiance dans ma capacité à contribuer à des projets pluridisciplinaires.

Sur le plan personnel, cette expérience m'a appris à travailler en équipe avec des profils variés, à communiquer clairement avec des collègues aux compétences différentes des miennes et à m'adapter à un environnement de recherche.

En résumé, ce stage m'a permis non seulement d'acquérir de nouvelles compétences techniques, mais aussi de développer des qualités humaines telles que l'adaptabilité, la pédagogie et le travail en équipe.

Références

1. Quanser. *QLabs Virtual Environment Documentation*.
<https://qlabs.quanserdocs.com/>
2. Quanser. *ACC Competition 2025*.
<https://github.com/quanser/ACC-Competition-2025>
3. Open Robotics. *ROS2 Documentation*.
<https://docs.ros.org/>
4. NVIDIA. *Isaac ROS Common Documentation*.
https://github.com/NVIDIA-ISAAC-ROS/isaac_ros_common
5. RobotWebTools. *rosbridge_suite and roslibjs Documentation*.
<https://robotwebtools.github.io/>

Outils d'IA utilisés :

1. OpenAI. ChatGPT. <https://chat.openai.com/>
2. Anthropic. Claude AI. <https://claude.ai/>
3. Google DeepMind. Gemini AI. <https://gemini.google.com/>

2 Rapport technique

2.1 Prise en main des environnements

La prise en main des environnements fournis par Quanser a représenté une étape essentielle pour comprendre la manière dont le simulateur QLab, le véhicule QCar2 virtuel et les packages ROS2 interagissent.

Le projet est basé sur l'*ACC Competition 2025 (American Control Conference)*, une compétition annuelle organisée par Quanser.

2.1.1 Environnements Docker utilisés par Quanser

Quanser fournit deux conteneurs Docker pour séparer la simulation et le développement :

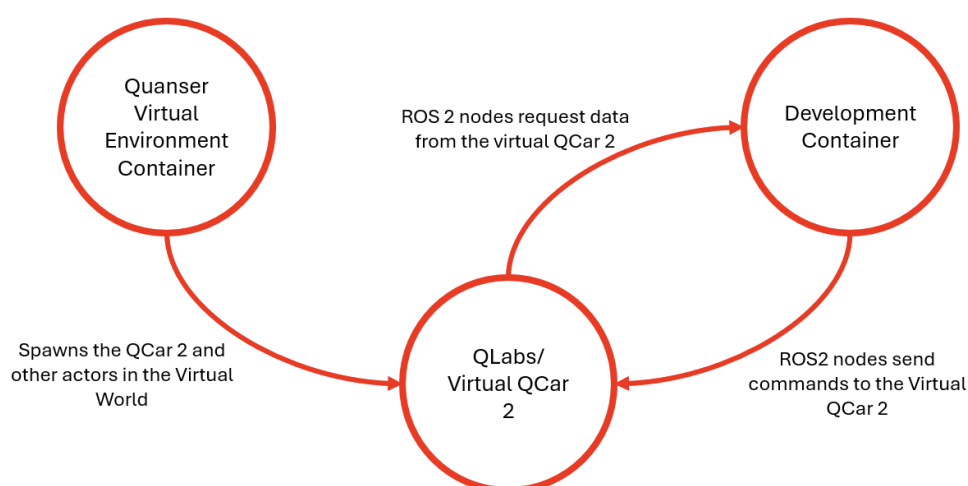


FIGURE 20 – Configuration de l'environnement virtuel QLab avec les conteneurs Docker.

- **Quanser Virtual Environment Container** : contient les scripts nécessaires à l'instanciation du QCar2 et des autres acteurs dans l'environnement virtuel. Le simulateur QLab, exécuté sur la machine hôte, s'appuie sur ce conteneur pour gérer la scène 3D et les entités simulées.

Par défaut, ces scripts sont intégrés directement dans l'image Docker, ce qui oblige à reconstruire l'image à chaque modification. Pour simplifier le développement, j'ai mis à jour le fichier Dockerfile et la commande de lancement du conteneur de manière à monter un dossier externe contenant mes scripts. Ainsi, les modifications de scripts sont prises en compte immédiatement sans avoir à reconstruire l'image, ce qui accélère considérablement les tests et les itérations.

```

Construire un nouveau image avec la commande : docker build -t my-virtual-qcar2
.

sudo docker run -rm -it \

-v $(pwd)/python:/home/qcar2_scripts/python \

-v $(pwd)/scripts:/home/qcar2_scripts/scripts \

-network host \

-name my-virtual-qcar2 \

my-virtual-qcar2 bash

```

Utilisation de `-v` permet de monter les dossiers `python` et `scripts` du répertoire courant de l'hôte dans `/home/qcar2_scripts/python` du conteneur à chaque lancement du conteneur.

- **Development Container** : contient l'environnement ROS2. Ce conteneur exécute l'environnement de développement basé sur ROS2. Quanser s'appuie sur l'infrastructure *Isaac ROS Common*, un projet proposé par NVIDIA qui fournit des Dockerfiles et des scripts préconfigurés pour la création d'environnements de développement ROS2 optimisés.

Dans notre dépôt, *Isaac ROS Common* est intégré en tant que sous-module Git, initialement configuré en version `release-2.1`. Cependant, lors de mon stage, le conteneur de développement a cessé de fonctionner. Après plusieurs sessions de débogage, j'ai conclu que le problème provenait probablement de cette ancienne version d'`isaac-ros-common`.

J'ai donc mis à jour le sous-module vers la version `release-3.2`, puis reconstruit le conteneur. Cette mise à jour a rétabli un environnement fonctionnel et stable, tout en assurant la compatibilité avec mes propres packages ROS2 développés.

```
cd /chemin/vers/isaac_ros_common
```

La commande pour lancer le conteneur Docker (ancienne version `release-2.1`) :

```
./scripts/run_dev.sh /chemin/vers/les/packages/ROS2
```

Nouvelle commande (nouvelle version `release-3.2`) :

```
./scripts/run_dev.sh --isaac_ros_dev_dir /chemin/vers/les/packages/ROS2
```

Dans ce conteneur, on retrouve :

- les packages ROS2 fournis par Quanser,
- mes propres packages développés durant le stage. C'est donc dans ce conteneur que j'ai implémenté, lancé et validé les différents nœuds ROS2 utilisés pour le développement du véhicule autonome.

2.1.2 Simulation d'un QCar2 dans QLabS

Le processus de lancement d'une simulation suit une séquence bien définie :

1. Création de l'environnement et instanciation du QCar2

Un script Python permet de générer l'environnement et un acteur QCar2 dans QLabS avec un identifiant unique.

```
# Spawn a QCar at the given initial pose
car2 = QLabSQCar2(qlabs)
car2.spawn_id(actorNumber=0,
               location=initialPosition,
               rotation=initialOrientation,
               scale=[.1, .1, .1],
               configuration=0,
               waitForConfirmation=True)
```

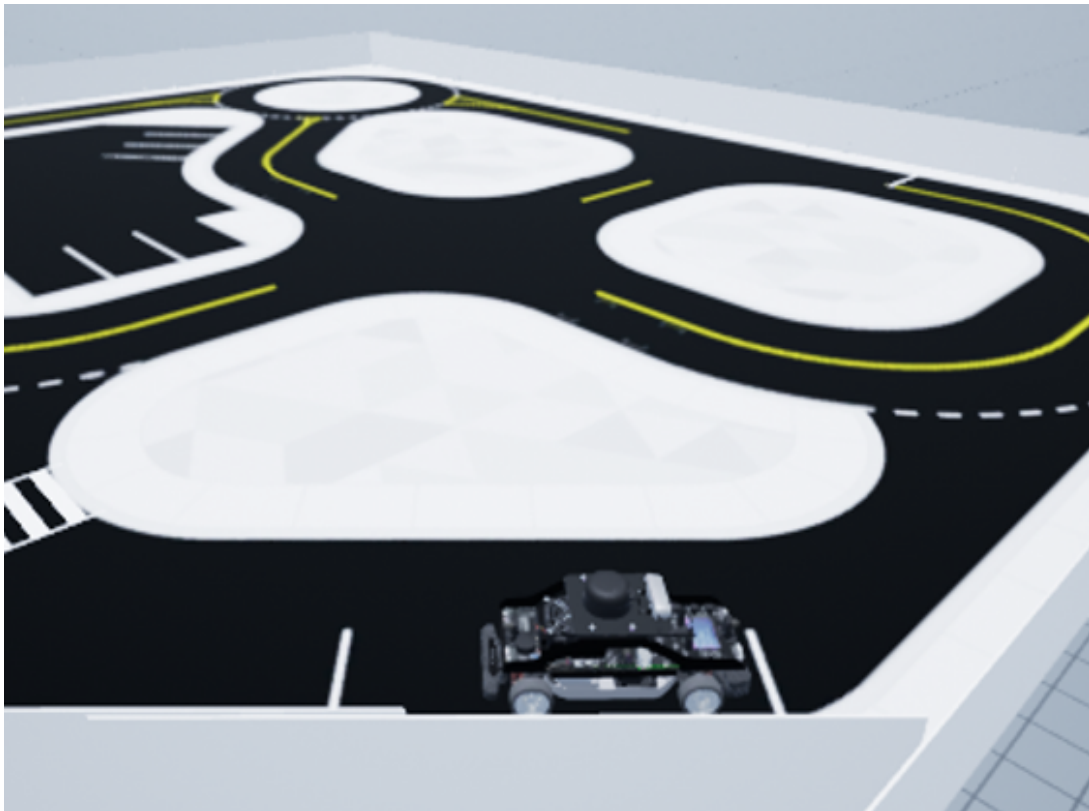


FIGURE 21 – QCar2 (à l'échelle 1/10) dans simulateur QLabS.

2. Lancement du modèle temps réel associé au QCar2

Le modèle temps réel est démarré :

```
rtModel = os.path.normpath(os.path.join(os.environ['RTMODELS_DIR'], 'QCar2/QCar2_Workspace_studio'))
```



```

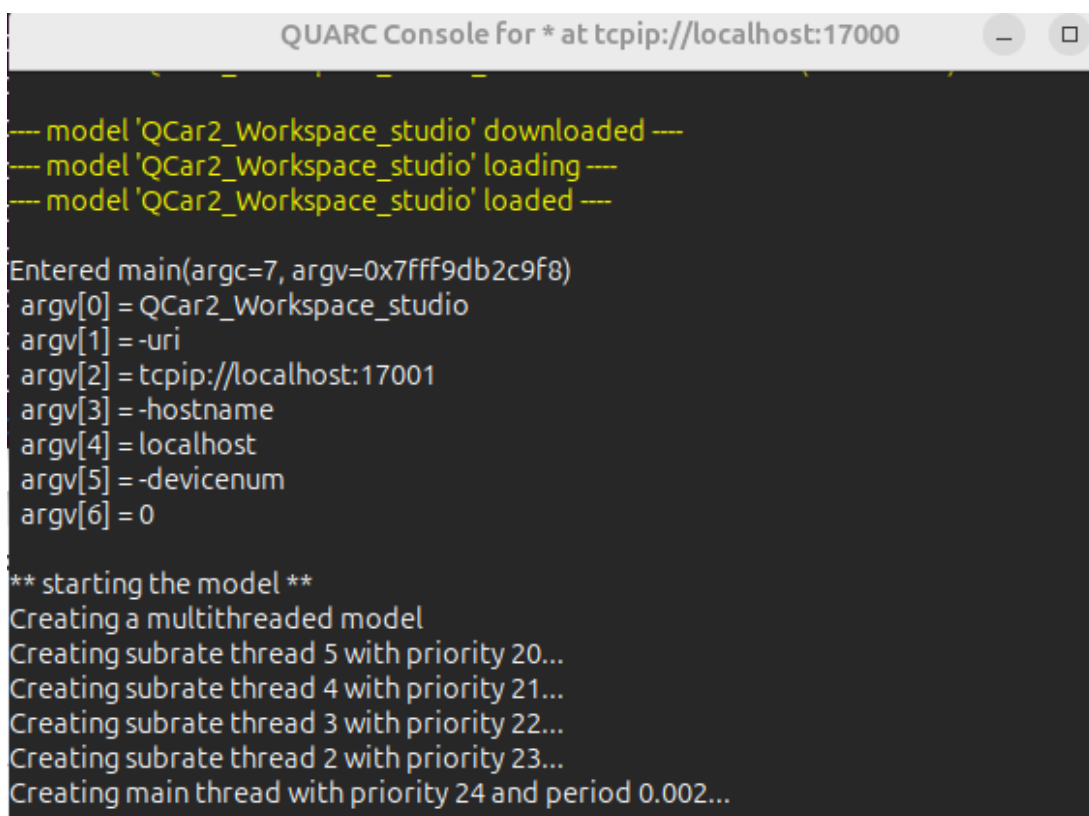
QLabsRealTime().start_real_time_model(rtModel)

def start_real_time_model(self, ...):
    ...
    cmdString="start_\"QLabs_{}_{}\"_\"_%QUARC_DIR%\quarc_run
        \"_D_-r_-t_tcpip://localhost:17000_\"{}_rt-win64\"_\"_
        uri_tcpip://localhost:{}_hostname_{}_devicenum_{}_{}
    ".format(modelName, actorNumber, modelName, self.
        _URIPort, QLABS_HOST_NAME, actorNumber,
        additionalArguments)
    ...

    os.system(cmdString)

```

Ce modèle ouvre automatiquement plusieurs ports TCP (18960-18969) pour diffuser les données simulées des capteurs (caméra, LIDAR, ...).



The screenshot shows a terminal window titled "QUARC Console for * at tcpip://localhost:17000". The output text is as follows:

```

--- model 'QCar2_Workspace_studio' downloaded ---
--- model 'QCar2_Workspace_studio' loading ---
--- model 'QCar2_Workspace_studio' loaded ---

Entered main(argc=7, argv=0x7fff9db2c9f8)
argv[0] = QCar2_Workspace_studio
argv[1] = -uri
argv[2] = tcpip://localhost:17001
argv[3] = -hostname
argv[4] = localhost
argv[5] = -devicenum
argv[6] = 0

** starting the model **
Creating a multithreaded model
Creating subrate thread 5 with priority 20...
Creating subrate thread 4 with priority 21...
Creating subrate thread 3 with priority 22...
Creating subrate thread 2 with priority 23...
Creating main thread with priority 24 and period 0.002...

```

FIGURE 22 – Console QUARC.

3. Vérification des ports ouverts par le modèle

Pour confirmer que le modèle s'exécute bien et identifier les ports utilisés, on peut lister les processus et filtrer ceux liés au QCar2. La commande suivante affiche uni-

quement les connexions réseau du fichier QCar2_Wor (correspondant au modèle lancé QCar2_Workspace_studio) :

```
sudo lsof -i -P -n | grep QCar2_Wor
```

- `lsof` (*list open files*) : commande qui liste tous les fichiers ouverts par les processus.
- `-i` filtre les fichiers liés à des connexions réseau (TCP/UDP)
- `-P` affiche les numéros de ports au lieu de noms de ports
- `-n` affiche directement les adresses numériques (sans la résolution DNS des adresses IP)
- `grep QCar2_Wor` : filtre la sortie pour ne garder que les lignes contenant QCar2_Wor, correspondant au modèle en temps réel QCar2_Workspace_studio.

La sortie de cette commande liste les différents ports TCP/UDP utilisés par le modèle temps réel.

```
QCar2_Wor 1105762 root 3u IPv4 1942692 0t0 TCP *:17001 (LISTEN)
QCar2_Wor 1105762 root 4u IPv4 1939028 0t0 UDP *:54372
QCar2_Wor 1105762 root 6u IPv4 1942693 0t0 TCP *:18960 (LISTEN)
QCar2_Wor 1105762 root 7u IPv4 1942694 0t0 TCP *:18969 (LISTEN)
QCar2_Wor 1105762 root 34u IPv4 1916447 0t0 TCP 127.0.0.1:56104->127.0.0.1:16999 (ESTABLISHED)
QCar2_Wor 1105762 root 38u IPv4 1943706 0t0 TCP 127.0.0.1:42548->127.0.0.1:18000 (ESTABLISHED)
QCar2_Wor 1105762 root 39u IPv4 1943709 0t0 UDP *:41106
QCar2_Wor 1105762 root 40u IPv4 1942695 0t0 TCP *:18961 (LISTEN)
QCar2_Wor 1105762 root 41u IPv4 1942696 0t0 TCP *:18962 (LISTEN)
QCar2_Wor 1105762 root 43u IPv4 1942697 0t0 TCP *:18963 (LISTEN)
QCar2_Wor 1105762 root 45u IPv4 1942698 0t0 TCP *:18964 (LISTEN)
QCar2_Wor 1105762 root 47u IPv4 1942699 0t0 TCP *:18965 (LISTEN)
QCar2_Wor 1105762 root 50u IPv4 1935527 0t0 TCP *:18968 (LISTEN)
QCar2_Wor 1105762 root 52u IPv4 1924869 0t0 TCP *:18967 (LISTEN)
QCar2_Wor 1105762 root 53u IPv4 1932723 0t0 TCP *:18966 (LISTEN)
```

FIGURE 23 – Sortie de la commande `lsof`.

4. Connexion ROS2 aux flux générés

Les bibliothèques ROS2 de Quanser sont configurées pour se connecter aux ports exposés par le modèle temps réel à l'aide du Quanser SDK.

Par exemple l'URI utilisé pour récupérer les données de LIDAR dans le cas de véhicule virtuel (port 18966) et le cas de véhicule physique :

```
if(device_type.compare("physical")==0){
    uri = "serial-cpu://localhost:1?baud='256000',word='8',
        parity='none',stop='1',flow='none',dsr='on'";
}
else if (device_type.compare("virtual")==0)
{
    uri = "tcpip://localhost:18966";
}
```

On obtient ce graphe dans `rqt_graph` une fois les nœuds lancés.

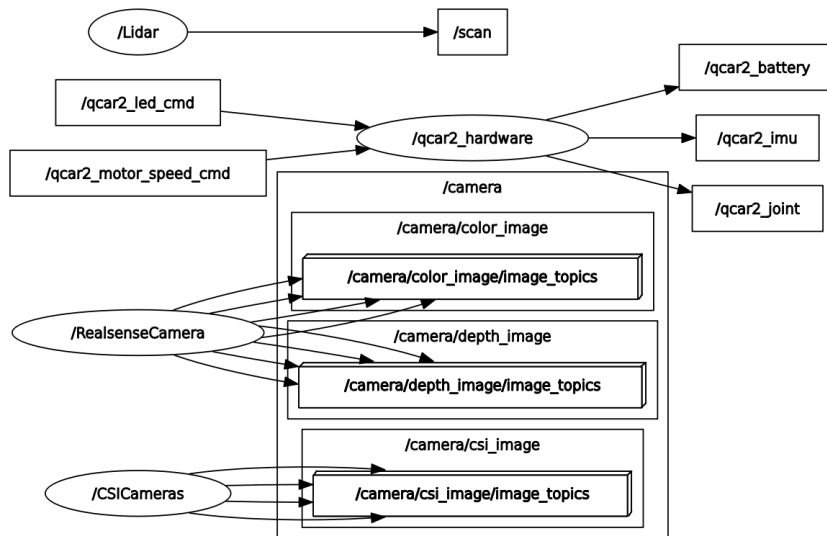


FIGURE 24 – Graphe des nœuds et topics ROS2 de QCar2.

2.2 Méthodologie de développement collaborative

J'ai d'abord créé un dépôt GitHub unique afin de centraliser le code. Dans le README.md, j'ai listé les étapes à suivre pour :



FIGURE 25 – Le fichier README.

- Cloner et configurer l'environnement : commande git pour cloner le dépôt utilisant le jeton GitHub, configuration à faire, installation des logiciels nécessaires, commandes à exécutées pour lancer les conteneurs Docker.
- Organisation des branches : présentation de la stratégie du développement utilisée Trunk-Based Development :

- développement dans des branches courtes,
- intégration rapide via *Pull Requests*,
- garantie que la branche `main` reste toujours fonctionnelle.
- Procédures Git : guide de développement détaillant le processus de développement étape par étape
 - Aller au GitHub Project et créer une "issue" liée à sa tâche.
 - Créer une branche temporaire pour travailler
 - Appliquer les bonnes pratiques : branche courte durée, commits fréquents.
 - Créer un *Pull Request* et fusionner dans la branche `main`.
 - Supprimer la branche temporaire.

2.3 Développement du véhicule autonome

2.3.1 Nœud `ekf.py` – Localisation

Ce nœud met en œuvre l'EKF et le GPS fournis par Quanser afin d'estimer la pose du QCar2.

Publication : estimation de pose `PoseStamped` (position, rotation) publiée sur `/ekf_pose` :

```
self.pub_pose = self.create_publisher(PoseStamped, "/ekf_pose", 50)

def publish_pose(self):
    msg = PoseStamped()
    # Recuperation des donnees de l'EKF
    ...
    self.pub_pose.publish(msg)
```

Abonnements :

Encodeurs (`/qcar2_joint`) : vitesse calculée en m/s :

```
self.sub_joint = self.create_subscription(JointState, '/qcar2_joint', self.joint_callback, 10)

def joint_callback(self, msg: JointState):
    self.motorTach = msg.velocity[0] * self.CPS_TO_MPS
```

IMU (`/qcar2_imu`) : récupérer la vitesse angulaire z utilisée pour la prédiction :

```
self.sub_imu = self.create_subscription(Imu, '/qcar2_imu', self.imu_callback, 10)

def imu_callback(self, msg: Imu):
    self.imu_data = msg.angular_velocity.z
```

```

# mise a jour de l'EKF
if self.imu_data:
    if self.gps.readGPS():
        # Recuperer les donnees GPS
        y_gps = ...

        self.ekf.update([self.motorTach, self.delta], dt,
                        y_gps, self.imu_data)

    # si le GPS ne fournit pas de nouvelles donnees de
    # mesure
    else:
        self.ekf.update([self.motorTach, self.delta], dt,
                        None, self.imu_data)

self.publish_pose()

```

2.3.2 Nœud lidar_occupancy_node.py – Occupancy Grid

Ce nœud transforme les mesures du LIDAR en une grille d'occupation exploitable par le contrôleur.

Publication : OccupancyGrid publiée sur /occupancy_grid :

```

self.occupancy_pub = self.create_publisher(OccupancyGrid, "/"
    occupancy_grid", 10)

```

```

def publish_occupancy_grid(self):
    msg = OccupancyGrid()
    ...
    self.occupancy_pub.publish(msg)

```

Abonnement : /scan : données du LIDAR, utilisées pour remplir la grille puis la publier :

```

self.subscription = self.create_subscription(LaserScan, '/'
    scan', self.scan_callback, 10)

```

```

def scan_callback(self, msg: LaserScan):
    ranges = np.array(list(msg.ranges))[:-1]
    angles = np.linspace(msg.angle_min, msg.angle_max, len(
        ranges))
    self.populate_occupancy_grid(ranges, angles)
    self.publish_occupancy_grid()

```

2.3.3 Nœud `vehicle_control_ros.py` – Contrôle et suivi de trajectoire

Ce nœud est responsable du contrôle du véhicule.

Publication : envoyer des commandes moteurs (*motor*, *steer*) sur `/qcar2_motor_speed_cmd` :

```
self.motor_pub = self.create_publisher(MotorCommands, "/
    qcar2_motor_speed_cmd", 10)

def send_control(self, motor, steer):
    msg = MotorCommands()
    ...
    self.motor_pub.publish(msg)
```

Abonnements :

`/ekf_pose` : récupérer la localisation du véhicule, utilisée pour le suivi de trajectoire et l'évitement :

```
self.ekf_sub = self.create_subscription(PoseStamped, '/
    ekf_pose', self.ekf_callback, 10)

def ekf_callback(self, msg: PoseStamped):
    pose = msg.pose
    x, y = pose.position.x, pose.position.y
    # calcul Stanley ou evitement selon occupancy grid
    ...
```

`/occupancy_grid` : récupérer l'*Occupancy Grid*, utilisé pour la détection d'obstacles :

```
self.occupancy_sub = self.create_subscription(OccupancyGrid,
    '/occupancy_grid', self.occupancy_callback, 10)

def occupancy_callback(self, msg: OccupancyGrid):
    self.occupancy_grid = np.array(msg.data).reshape(msg.info
        .height, msg.info.width)
```

Fonctionnalités principales :

1. **Utilisation de Stanley Controller** : pour suivre une trajectoire planifiée :

```
# generer la trajectoire
roadmap = SDCSRoadMap()
self.waypointSequence = roadmap.generate_path(self.
    nodeSequence)

# initialiser le controleur avec le trajectoire
self.stanleyController = StanleyController(
```

```

        waypoints=self.waypointSequence
    )

```

Dans la boucle de contrôle :

```

motor = self.speedController.update(...)

delta = self.stanleyController.update(x, y, yaw, v)
...
self.send_control(motor, delta)

```

2. Évitement d'obstacles grâce à l'Occupancy Grid :

Dans la boucle de contrôle, avant d'envoyer la commande de contrôle :

```

if self.avoid_obstacle_occupancy_grid(...):
    return # commande d'évitement envoyée

```

3. Détection de collision – check_collision()

Cette méthode prend en entrée :

- deux cellules : la position du véhicule et la cible actuelle (un point devant le véhicule sur la trajectoire),
- une marge de sécurité.

Le principe de cette méthode : trace les lignes sur la grille entre la position du véhicule et la cible et récupère les cellules traversées, pour chaque ligne elle applique un décalage latéral correspondant à la marge, et vérifie si ces lignes croisent une cellule occupée dans la grille.

```

def check_collision(self, cell_a, cell_b, margin=0):
    for i in range(-margin, margin + 1):
        cell_a_margin = (cell_a[0], cell_a[1] + i)
        cell_b_margin = (cell_b[0], cell_b[1] + i)

        for cell in self.utils.traverse_grid(cell_a_margin,
            cell_b_margin):
            if self.occupancy_grid[cell] == self.IS_OCCUPIED:
                return True
    return False

```

4. Génération d'une nouvelle cible

Lorsque check_collision() détecte une collision :

- L'algorithme cherche une cible locale alternative autour de la trajectoire courante.
- Pour cela, il génère plusieurs points candidats décalés latéralement (de gauche à droite de la cible d'origine).
- Chaque point est testé avec check_collision().

- Le premier qui est libre est choisi comme nouvelle cible.

Une fois ce nouveau point trouvé :

- le contrôleur calcule l'angle pour s'orienter vers ce point,
- une commande est envoyée avec `send_control()`.

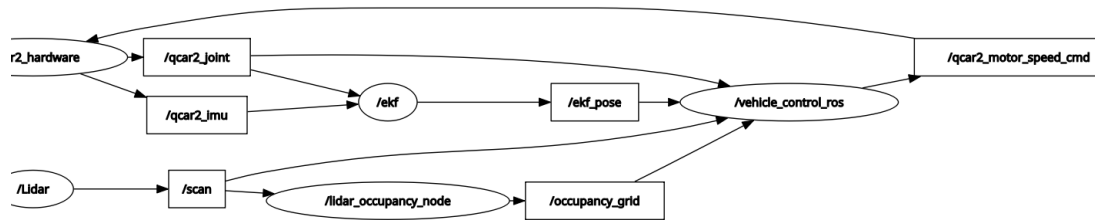


FIGURE 26 – Graphe des échanges de données entre les nœuds.

Légende des symboles :

- Ellipses ovales : représentent les nœuds ROS2.
- Rectangles : représentent les topics ROS2.
- Flèches : indiquent la direction du flux de données de nœud.



FIGURE 27 – Evitement d'obstacle.

L'image ci-dessus illustre la prise de décision du contrôleur. Le point vert représente la position actuelle du véhicule. Le point bleu correspond à la cible initiale sur la trajectoire planifiée (points rouges). Les points bleus indiquent les mesures LIDAR, et les cellules blanches représentent les cellules occupées de l'*Occupancy Grid*. Comme le chemin vers la cible est obstrué, le contrôleur génère une nouvelle cible locale (point jaune), permettant au véhicule de contourner l'obstacle.

2.4 Développement d'une interface utilisateur (application web)

2.4.1 Architecture générale

L'interface utilisateur web repose sur deux composants principaux :

- Un serveur Express.js : utilisé pour retourner les pages web (HTML, JavaScript, CSS). Il ne contient pas de logique complexe, mais sert à héberger et distribuer l'application.
- `rosbridge-suite` et `roslibjs` : la communication entre l'application et ROS2 s'effectue grâce au paquet `rosbridge-server` de `rosbridge-suite`, qui expose les nœuds, topics et services ROS2 via WebSocket, et à la bibliothèque JavaScript `roslibjs`, qui permet de récupérer l'information des nœuds, publier et souscrire directement aux topics depuis le navigateur.

2.4.2 Architecture du code

Architecture Backend

`server.js` : script principal pour configurer Express.js

- Il sert les fichiers statiques (HTML, CSS, JavaScript)
- Il définit deux routes :
 - `/` : retourne `index.html` (Dashboard).
 - `/settings` : retourne `settings.html` (page de configuration).

Architecture Frontend

Le frontend emploie une architecture JavaScript modulaire avec une séparation claire des responsabilités :

1. `main.js` : point d'entrée de l'application et initialisation.
2. `config.js` : gestion de la configuration et constantes.
3. `ros-connection.js` : gestion de la connectivité au serveur `rosbridge`.
4. `node-list.js` : interface d'affichage et de sélection de nœuds.
5. `node-parameters.js` : gestion des informations et mis à jour des paramètres de nœuds sélectionnés.
6. `topic-subscriptions.js` : abonnements aux topics publiés par le nœud sélectionné.
7. `message-visualizations.js` : Système de visualisation de messages reçu basé sur les classes.
8. `event-handlers.js` : Gestion globale des événements.
9. `settings-page.js` : Sauvegarde des préférences utilisateur et configuration dans `localStorage`.

2.4.3 Intégration de ROS2 avec roslib.js

Gestion des connexions

- Connexion WebSocket ROSBridge.
- Surveillance du statut de connexion avec indicateurs visuels.
- Paramètres de connexion configurables (hôte, port).

```
// recuperer les parametres stockes dans localStorage
const ip = ...
const port = ...
const url = `ws://${ip}:${port}`;
ros = new ROSLIB.Ros({
  url: url
});
```

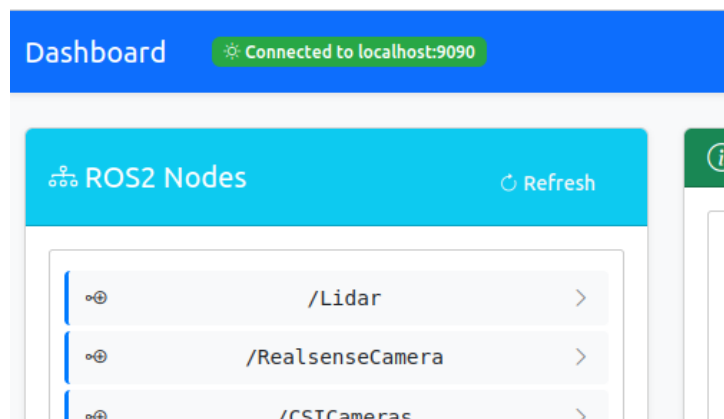


FIGURE 28 – Indicateur visuel de statut de connexion.

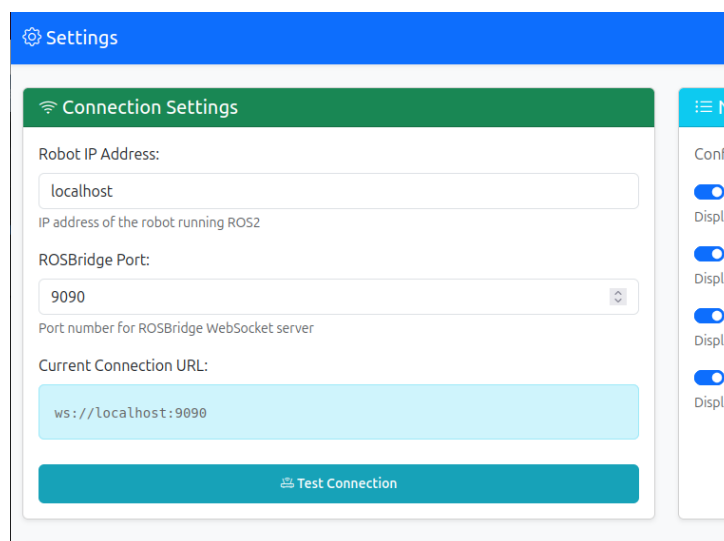


FIGURE 29 – Paramètres de connexion ROSBridge.

Gestion des nœuds

- Sélection interactive de nœuds : Interface de sélection basée sur les clics
- Affichage d'informations de nœuds :
 - Liste des topics abonnés
 - Liste des topics publiés
 - Services disponibles
 - Paramètres éditables

La récupération des nœuds s'effectue via un appel de service. Le service est fourni par le nœud `rosapi` du serveur Rosbridge.

```
const nodesClient = new ROSLIB.Service({
  ros: ros,
  name: '/rosapi/nodes',
  serviceType: 'rosapi/Nodes'
});
const request = new ROSLIB.ServiceRequest({});
nodesClient.callService(request, function(result) {
  ...
});
```

De même pour la récupération des informations du nœud :

```
const detailClient = new ROSLIB.Service({
  ros: ros,
  name: '/rosapi/node_details',
  serviceType: 'rosapi/NodeDetails'
});
...
```

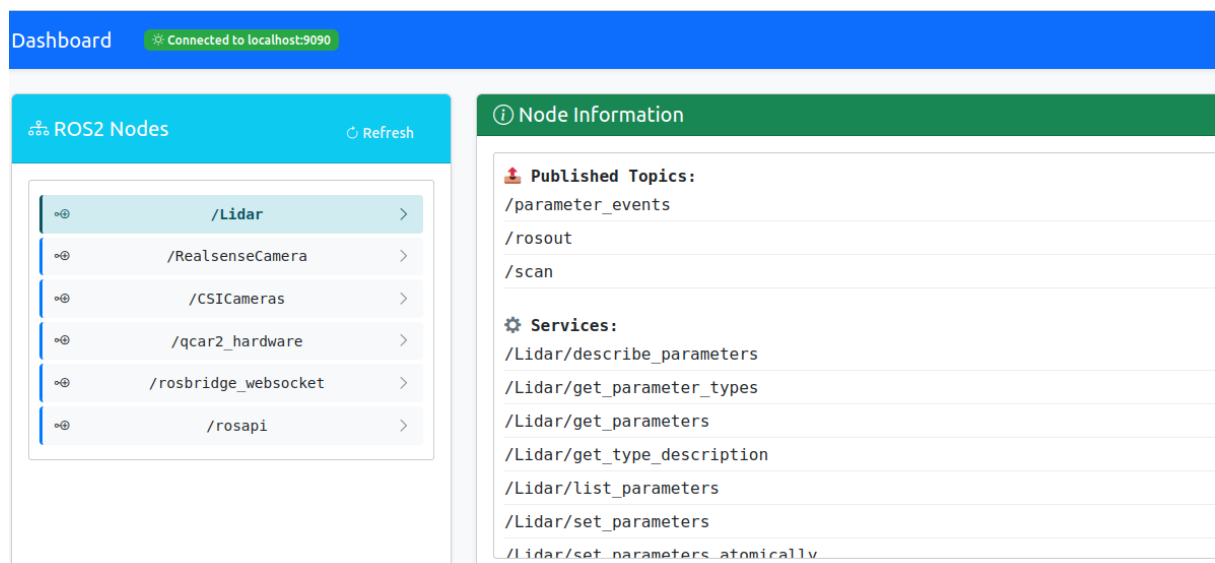


FIGURE 30 – Affichage d'informations de nœuds sélectionnés.

La mise à jour des paramètres :

```
const param = new ROSLIB.Param({
  ros: ros,
  name: paramName
});

param.set(newValue, function() {
  ...
});
```

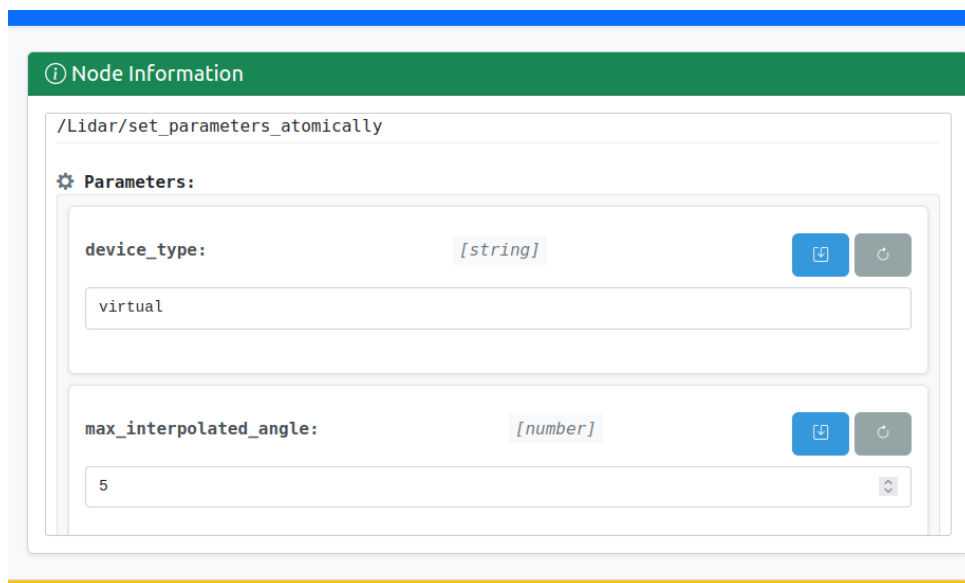


FIGURE 31 – Mise à jour des paramètres du nœud sélectionné.

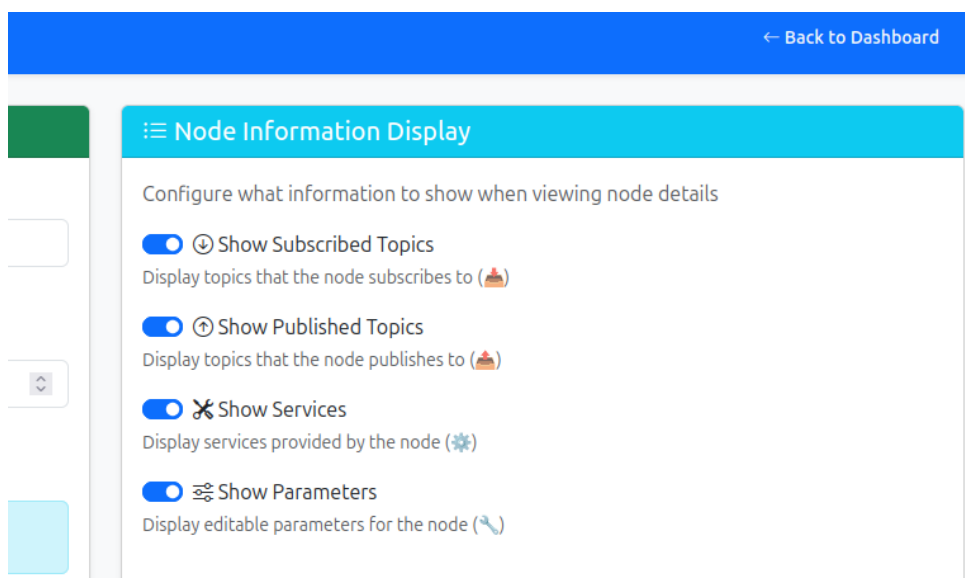


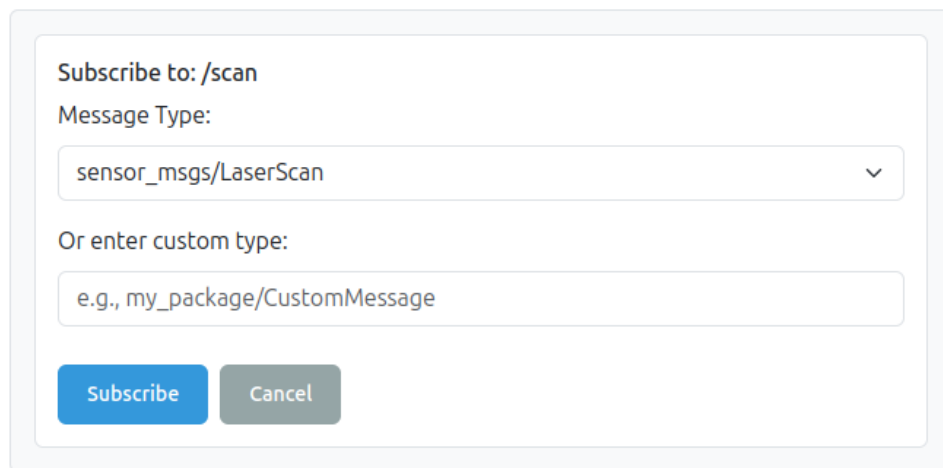
FIGURE 32 – Paramètres d'affichage du contenu des informations du nœud.

Gestion d'Abonnement aux Topics

- Sélection de topic de nœud sélectionné à s'abonner et type de message.
- Gestion des topics abonnés et désabonnement.

```
// stocker les abonnements aux topics actifs
let activeSubscriptions = new Map();

const listener = new ROSLIB.Topic({
  ros: ros,
  name: topicName,
  messageType: messageType
});
listener.subscribe(function(message) {
  // visualiser le message reçu
})
activeSubscriptions.set(topicName, {
  listener: listener,
  messageType: messageType,
  startTime: new Date()
});
```



Subscribe to: /scan

Message Type:

sensor_msgs/LaserScan

Or enter custom type:

e.g., my_package/CustomMessage

Subscribe Cancel

FIGURE 33 – Sélection de type de message.

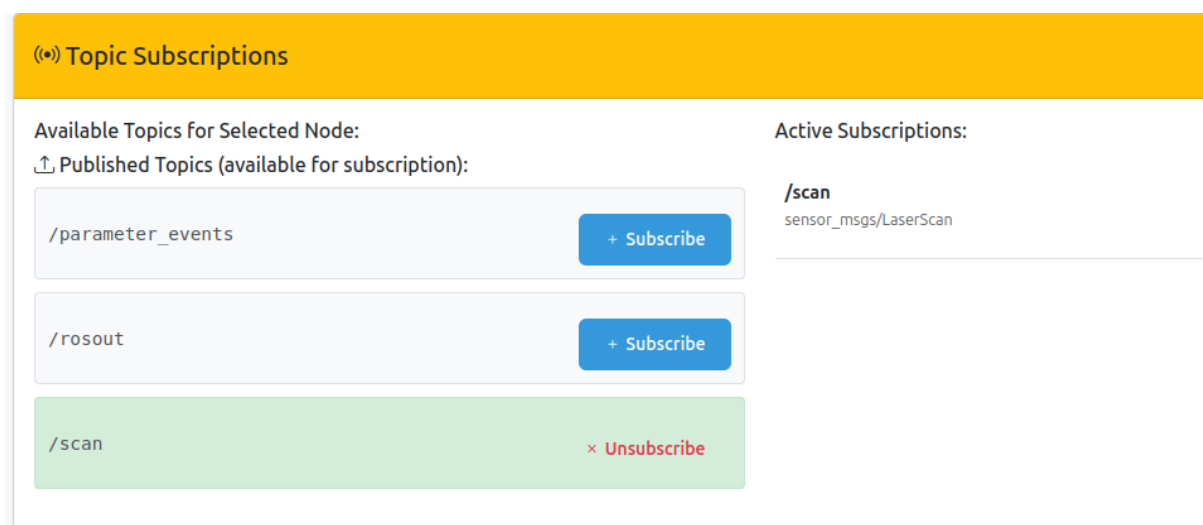


FIGURE 34 – Liste des topics de nœud courant et les abonnement actifs.



FIGURE 35 – Visualisation de message du topic /scan.

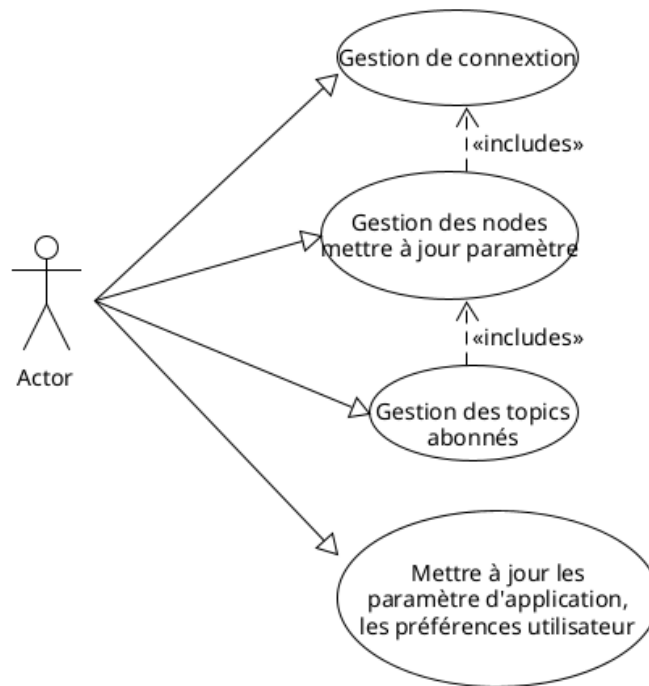


FIGURE 36 – Diagramme de cas d'utilisation.

2.4.4 Structure de visualisation de message

Le système de visualisation de messages utilise une conception orientée objet avec héritage :

Classe de Base : `MessageVisualization`.

- Constructeur : Prend les paramètres "vizId", "topicName" et "messageType"
- Méthodes abstraites :
 - `generateInitialHTML()` : générer un conteneur HTML pour ce type de message.
 - `updateWithMessage(message)` : mettre à jour les éléments du conteneur avec le nouveau message.
- Méthodes utilitaires :
 - `getElement(elementId)` : récupération des éléments DOM
 - `updateElement(elementId, content)` : mises à jour du contenu HTML interne
 - `updateElementText(elementId, text)` : mises à jour du contenu texte

Pour l'instant, il ne prend en charge que 3 types de message, les autres afficheront le message au format JSON par défaut.

- `TwistVisualization` : `geometry_msgs/Twist`.
- `LaserScanVisualization` : `sensor_msgs/LaserScan`.
- `ImageVisualization` : `sensor_msgs/Image`.

- `DefaultVisualization` : Affichage de messages génériques au format JSON.

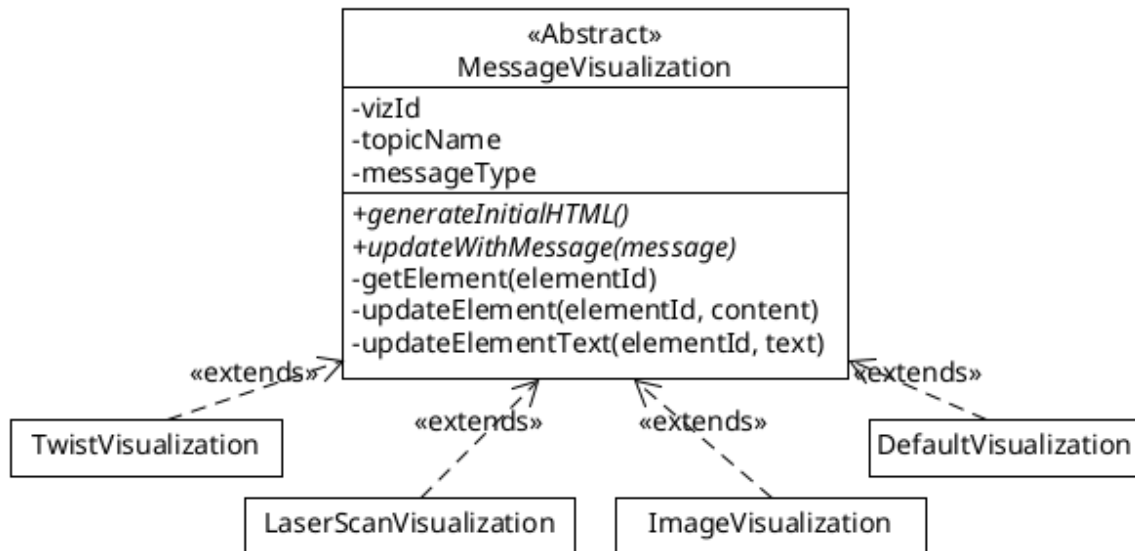


FIGURE 37 – Diagramme de classe de structure de visualisation de message.

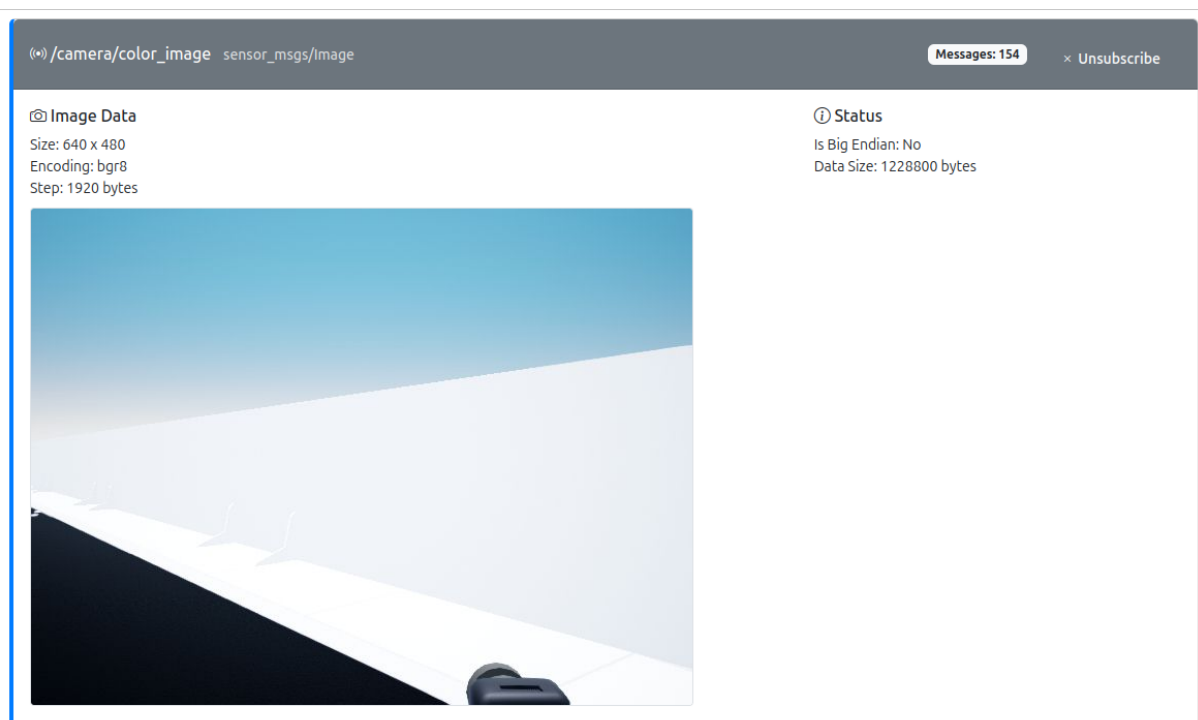


FIGURE 38 – Visualisation de message du topic `/camera/color_image` (classe `ImageVisualization`).